

1 Apresentação

Este projeto vale **5,0 pontos**, correspondentes a **50% da nota** da disciplina, e consolida o conteúdo de linguagens formais, autômatos finitos, autômatos com pilha e Máquina de Turing.

O objetivo é construir um pequeno sistema aplicado de validação formal contendo três reconhecedores, um para cada nível da hierarquia de Chomsky:

1. um reconhecedor de **Linguagem Regular** (LR);
2. um reconhecedor de **Linguagem Livre de Contexto** (LLC);
3. um reconhecedor de **Linguagem Recursiva** (R).

O trabalho tem aplicação prática, mas mantém o foco em Teoria da Computação. A meta é comparar, em código que vocês mesmos escrevem, a hierarquia

$$LR \subsetneq LLC \subsetneq R.$$

- **Equipes:** trios.
- **Plataforma de submissão:** Blackboard.
- **Repositório GitHub público:** link postado no Blackboard.
- **Apresentação ou vídeo** (10 a 12 minutos) no **YouTube como unlisted**: link postado no Blackboard.
- **Prazo final:** dia 10.

2 Contexto: por que três níveis?

Toda vez que um software decide se uma entrada é válida, ele está reconhecendo uma linguagem formal. Validadores de formulário (CPF, e-mail, IPv4), parsers de protocolo (TCP, HTTP, BGP), verificadores de logs e mecanismos de detecção de intrusão fazem isso o tempo todo.

Esses validadores diferem fundamentalmente no **tipo de problema** que conseguem resolver:

- Validar o formato de um CPF é um problema **regular**, decidido por um DFA com poucos estados.
- Verificar que os parênteses de uma fórmula estão balanceados é um problema **livre de contexto**, que exige uma pilha.
- Comparar se duas cadeias são idênticas (formato $w\#w$) ou contar três grupos iguais ($a^n b^n c^n$) são problemas **recursivos**, que exigem uma MT.

O projeto pede a construção dos três reconhecedores, no mesmo tema aplicado, para que a equipe sinta na prática a diferença de poder computacional entre eles.

3 Temas disponíveis

Cada trio escolhe **um** dos dois temas. Os três níveis (LR, LLC, R) são obrigatórios, independentemente do tema escolhido.

3.1 Tema 1: Validador de cadastro, fórmulas e integridade

Classe	Aplicação	Modelo esperado
LR	Uma das opções: CPF no formato <code>ddd.ddd.ddd-dd</code> , data no formato <code>YYYY-MM-DD</code> , ou IPv4 simplificado (<code>d.d.d.d</code> com 1 a 3 dígitos por bloco)	Regex e DFA ou NFA
LLC	Parênteses, colchetes ou chaves balanceadas em uma expressão simbólica	Pilha, PDA ou GLC
R	Cópia de cadeia: $L = \{w\#w \mid w \in \{0,1\}^*\}$	Máquina de Turing

No CPF, validar apenas o formato textual, não os dígitos verificadores. No IPv4, validar apenas o formato (quatro blocos de 1 a 3 dígitos separados por ponto), **não** a faixa de 0 a 255 de cada bloco: faixas numéricas exigiriam aritmética e descaracterizariam o DFA.

Exemplos:

- LR: 123.456.789-00 aceito; 12.34.56-78 rejeitado.
- LLC: $((x+y)*z)$ aceito; $((a+b))$ rejeitado.
- R: 101#101 aceito; 101#100 rejeitado.

3.2 Tema 2: Validador de logs e protocolos

Classe	Aplicação	Modelo esperado
LR	Sequência de eventos que começa com LOGIN, seguido de AUTH, seguido de zero ou mais REQUEST, terminando com LOGOUT	DFA
LLC	Blocos aninhados de transação: cada BEGIN precisa de um END correspondente, podendo aninhar	Pilha, PDA ou GLC
R	Trio balanceado de eventos: $L = \{OPEN^n COMMIT^n CLOSE^n \mid n \geq 1\}$ ou, alternativamente, espelhamento de transação $L = \{w\#w \mid w \in \{R,W\}^*\}$ (sequência de leituras/escritas seguida do mesmo trecho, separados por #)	Máquina de Turing

Exemplos:

- LR: LOGIN AUTH REQUEST REQUEST LOGOUT aceito; LOGIN LOGOUT AUTH rejeitado.
- LLC: BEGIN BEGIN END END aceito; BEGIN END END rejeitado.
- R: OPEN OPEN COMMIT COMMIT CLOSE CLOSE aceito; OPEN COMMIT CLOSE CLOSE rejeitado.

4 Exigências para cada reconhecedor

Para cada uma das três linguagens, a equipe deve apresentar:

1. descrição em português da linguagem;
2. definição formal em notação matemática ($\{w \in \Sigma^* \mid \dots\}$ ou equivalente);
3. alfabeto utilizado;
4. exemplos de cadeias aceitas e rejeitadas;
5. modelo computacional escolhido, com **tabela de transição completa** (DFA, MT) ou **transições principais com evolução da pilha** (PDA), acompanhado de diagrama;
6. implementação em Python;
7. testes automatizados;
8. execução passo a passo de uma cadeia aceita e uma cadeia rejeitada;
9. número de passos consumidos pelo programa em cada teste.

Implementação manual obrigatória. Em cada um dos três níveis, o reconhecedor é um **simulador do autômato escolhido** (DFA, PDA ou MT), com estados, alfabeto(s), estado inicial, estados finais e tabela de transição declarados **explicitamente como dados** no código (por exemplo, como dicionários ou conjuntos), e não como uma sequência de `if/else`. O programa deve aplicar a função de transição símbolo a símbolo (ou movimento a movimento, no caso da MT) sobre a entrada.

Sobre o uso de `re`. O módulo `re` do Python (expressões regulares) **não pode ser usado como reconhecedor principal** em nenhum dos três níveis. No nível LR, `re` pode aparecer **apenas como comparação opcional** (item de bônus), nunca como engine de aceitação/rejeição. Bibliotecas auxiliares como Graphviz, pytest ou matplotlib são bem-vindas para diagramas e testes, mas **não substituem** o reconhecedor.

Definição operacional de “passo”. Para fins de contagem e comparação entre níveis, **um passo é uma aplicação da função de transição do modelo**, ou seja:

- DFA/NFA: cada leitura de um símbolo da entrada com mudança de estado;
- PDA: cada transição, contando empilhamento e desempilhamento;
- MT: cada movimento da cabeça (leitura, escrita, deslocamento) sobre a fita.

O número de passos **não é** o número de iterações do laço Python nem o número de linhas executadas. Cada reconhecedor deve incrementar um contador exclusivamente nesses eventos.

5 Bateria de testes

A bateria deve conter, para cada uma das três linguagens:

- 3 cadeias aceitas;
- 3 cadeias rejeitadas.

Total: 18 cadeias. Incluir casos de borda quando fizer sentido (cadeia vazia, cadeia de tamanho 1, cadeia logo no limite da regra).

O programa deve:

- imprimir uma **tabela esperado vs obtido** para cada teste;
- informar o **número de passos** executados pelo reconhecedor em cada cadeia.

O relatório deve detalhar a execução passo a passo de **uma cadeia aceita e uma cadeia rejeitada por linguagem**.

Testes adicionais durante a avaliação (cadeias-surpresa). O professor rodará os reconhecedores com cadeias adicionais, não presentes nos arquivos entregues. Essas cadeias seguirão o mesmo alfabeto e a mesma definição formal de cada linguagem. **Parte da nota do bloco “Testes e análise de resultados” (até 0,25 ponto) é reservada ao desempenho do reconhecedor nessas cadeias-surpresa.** O objetivo é verificar se o programa reconhece a linguagem de forma geral, não apenas os exemplos da bateria.

6 Forma de entrega

A entrega é composta por três partes:

1. **Repositório GitHub público** com o código.
2. **Relatório técnico** em PDF.
3. **Vídeo de apresentação** no YouTube como unlisted, com duração de 10 a 12 minutos.

Os links do repositório e do vídeo devem ser postados no Blackboard até o **dia 10**.

6.1 Estrutura do repositório

```
projeto/  
  README.md  
  requirements.txt  
  src/  
    regular.py          (reconhecedor LR, executável independentemente)  
    livre_contexto.py   (reconhecedor LLC, executável independentemente)  
    recursiva.py        (reconhecedor R, executável independentemente)  
    testes.py           (lê os arquivos testes/*.txt e roda contra os 3 reconhecedores)  
  testes/  
    testes_regular.txt  
    testes_livre_contexto.txt  
    testes_recursiva.txt  
  diagramas/  
    dfa_regular.(png|svg|pdf|tex)  
    pda_livre_contexto.(png|svg|pdf|tex)  
    mt_recursiva.(png|svg|pdf|tex)  
  relatorio/  
    relatorio.pdf
```

Cada reconhecedor deve ter um modo de execução autônomo (por exemplo, `python src/regular.py "123.456.789-00"`), além de ser usado por `testes.py`. O README deve permitir rodar a bateria completa em **um único comando**.

Diagramas podem ser entregues em PNG, SVG, PDF ou código-fonte TikZ/Graphviz (`.tex`, `.dot`). Fotos de quadro/papel não são aceitas.

6.2 Relatório técnico (PDF, 4 a 7 páginas)

Estrutura mínima:

1. Introdução e contexto aplicado.
2. Linguagem Regular escolhida: descrição, definição formal, modelo, exemplos.
3. Linguagem Livre de Contexto escolhida: descrição, definição formal, modelo, exemplos.

4. Linguagem Recursiva escolhida: descrição, definição formal, modelo, exemplos.
5. Modelos formais e transições (tabelas e diagramas dos três reconhecedores).
6. Implementação (visão geral do código, decisões principais).
7. Testes e resultados (tabela esperado vs obtido, número de passos).
8. Comparação entre os três níveis da hierarquia.
9. Conclusão e limitações.

6.3 Vídeo de apresentação

Duração: 10 a 12 minutos.

Deve conter:

1. contexto aplicado;
2. explicação das três linguagens escolhidas;
3. modelos formais usados;
4. demonstração do programa rodando a bateria;
5. resultados dos testes;
6. comparação entre LR, LLC e R.

Divisão de fala obrigatória. Cada integrante do trio deve apresentar pelo menos um dos três níveis (LR, LLC ou R), incluindo o modelo formal, a implementação e ao menos um teste. Os blocos de abertura (contexto) e fechamento (comparação entre níveis) podem ser distribuídos livremente. A ausência de fala de algum integrante implica perda da nota individual de apresentação.

O vídeo não deve ser uma leitura linha por linha do código.

7 Bônus opcional

Até 0,5 ponto extra, respeitando o teto de 5,0, para itens **bem implementados e bem interpretados**:

- interface simples em Streamlit ou Gradio para testar cadeias nos três reconhecedores;
- visualização animada do autômato, da pilha ou da fita da MT;
- minimização do DFA do nível LR;
- comparação direta entre o DFA manual e o módulo `re` do Python na linguagem LR;
- implementação de uma segunda MT (por exemplo, $\{a^n b^n c^n\}$, palíndromos binários, ou a MT do tema que a equipe não escolheu);
- discussão sobre o crescimento do número de passos em função do tamanho da entrada;
- prova de não-regularidade via Lema do Bombeamento aplicada à linguagem LLC ou R escolhida;
- uso opcional de Lark, pyformlang ou JFLAP apenas como comparação.

O Lema do Bombeamento é estritamente bônus, não é necessário usá-lo na parte obrigatória.